

机器学习与数据挖掘第二次大作业

小组成员

实验原理

注意力

注意力机制允许模型在处理每个词时关注它和其他词的关系。在多头注意力机制中，多个注意力头并行计算，可以同时学习不同的依赖关系。

Q,K,V

在自注意力机制中，每个输入向量会被转化为三个向量。

Q是查询向量，表示模型希望关注的信息，用来衡量和其它所有位置上的信息的相关性。它通常与当前位置（例如解码阶段的当前位置）或者整个序列（编码阶段）有关的信息相对应。

K是键向量，用于匹配Q向量。它是输入序列中每个位置的信息的“索引”或“查找表”，可以理解为每个位置的标识特征。

V是值向量，表示序列中每个位置的实际内容。

GPT架构

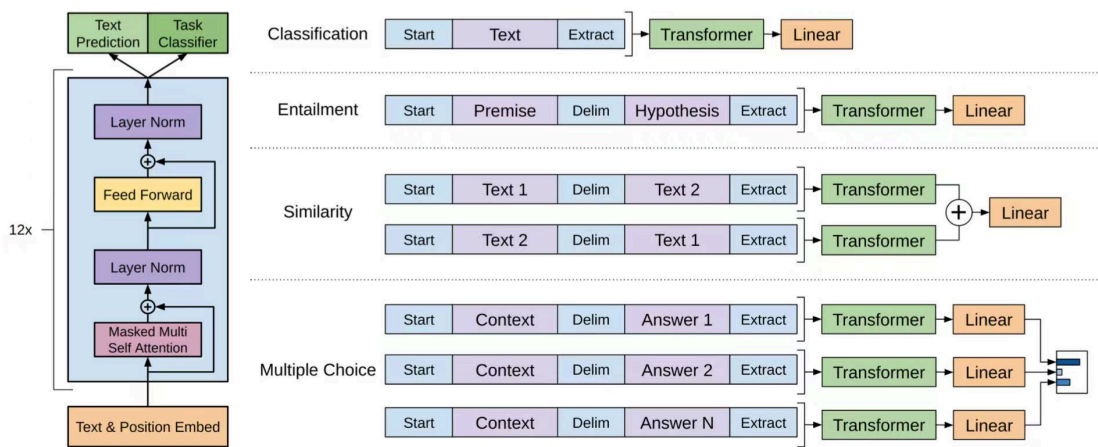


Figure 1: **(left)** Transformer architecture and training objectives used in this work. **(right)** Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

GPT的结构包括单词编码、位置编码、多个多头注意力层、多个前馈神经网络层、linear变换和Softmax输出。linear变换用于将维度转换为词表的个数，Softmax用于预测下一个单词。

实验准备

实验环境

本次实验使用 python3.9 + pytorch2.5.1 的环境进行任务。

数据预处理

数据处理

谢谢你所做的一切
你开心就好
开心
嗯因为你的心里只有学习
某某某，还有你
这个某某某用的好

你们宿舍都是这么厉害的人吗
眼睛特别搞笑这土也不好捏但就是觉得挺可爱
特别可爱啊

今天好点了吗？
一天比一天严重
吃药不管用，去打一针。别拖着

是的。下辈子想做只萤火虫
可是萤火虫太容易被抓了还是改一个吧
不，我只想奋不顾身扑火

我们使用的数据集是一个.txt文件，由一系列对话组成。每组对话用一个空行隔开，组内每句话占一行。

```
with open('train.txt', 'r', encoding='utf-8') as f:
    lines = f.readlines()

train_datas = []
temp_data = ''
for line in lines:

    if line != '\n':
        line = line.strip()
        temp_data += (line + '\t')
    else:
        train_datas.append(temp_data)
        temp_data = ''
```

为了方便后续处理，我们将数据集中的换行符替换成制表符。这样，每组对话可以在一行中显示完，组内的每一句话之间用制表符隔开。

```
谢谢你所做的一切    你开心就好    开心 嗯因为你的心里只有学习    某某某，还有你 这个某某某用的好
你们宿舍都是这么厉害的人吗    眼睛特别搞笑这土也不好捏但就是觉得挺可爱    特别可爱啊
今天好点了吗？ 一天比一天严重 吃药不管用，去打一针。别拖着
是的。下辈子想做只萤火虫 可是萤火虫太容易被抓了还是改一个吧 不，我只想奋不顾身扑火
加油，三月动起来，五月笑起来 正解你为什么就那么厉害呢 哈哈，没办法，智商就是这么高 你这是要开始得瑟了吗！好啦！你最厉害！ 哈哈哈哈哈
好身材，秀出来，哈哈其实我早就的，不介意的
```

由于电脑算力限制，为了避免模型训练用时过长，我们去掉了长度大于200的对话，并将符合条件的对话写入新的.txt文件。

```
def make_data(datas):
    train_datas = []
    for data in datas:
        data=data.strip()
        train_data = [i if i!='\t' else "<sep>" for i in data]+['<sep>']
        train_datas.append(train_data)

    return train_datas
```

为了方便生成字典，我们将所有的制表符替换成 <sep> .，并在末尾添加一个 <sep>

生成字典

```
def get_dict(datas):
    word_count = {}
    for data in datas:
        data = data.strip().replace('\t','_')
        for word in data:
            word_count.setdefault(word,0)
            word_count[word]+=1
    word2id = {"<pad>":0,"<unk>":1,"<sep>":2}

    temp = {word: i + len(word2id) for i, word in enumerate(word_count.keys())}
    word2id.update(temp)
    id2word=list(word2id.keys())
    return word2id,id2word
```

在这里构建了word2id字典和id2word列表。

```
{ "word2id": { "<pad>": 0, "<unk>": 1, "<sep>": 2, "\u0c22": 3, "\u4f60": 4, "\u6240": 5, "\u505a": 6, "\u7684": 7, "\u4e00": 8,
< "\u5207": 9, "\u5f00": 10, "\u5fc3": 11, "\u5c31": 12, "\u597d": 13, "\u55ef": 14, "\u56e0": 15, "\u4e3a": 16, "\u91cc": 17, "\u53ea":
< 18, "\u6709": 19, "\u5bb6": 20, "\u4e60": 21, "\u67d0": 22, "\uff0c": 23, "\u8fd8": 24, "\u8fd9": 25, "\u4e2a": 26, "\u7528": 27,
< "\u4eec": 28, "\u5bbf": 29, "\u820d": 30, "\u90fd": 31, "\u662f": 32, "\u4e48": 33, "\u5389": 34, "\u5bb3": 35, "\u4eba": 36,
< "\u5417": 37, "\u773c": 38, "\u775b": 39, "\u7279": 40, "\u522b": 41, "\u641e": 42, "\u7b11": 43, "\u571f": 44, "\u4e5f": 45,
< "\u4e0d": 46, "\u634f": 47, "\u4f46": 48, "\u89c9": 49, "\u5f97": 50, "\u633a": 51, "\u53ef": 52, "\u7231": 53, "\u554a": 54,
< "\u4eca": 55, "\u5929": 56, "\u70b9": 57, "\u4e86": 58, "\uff1f": 59, "\u6bd4": 60, "\u4e25": 61, "\u91cd": 62, "\u5403": 63,
<
```

在word2id字典里，存储了每个字和对应的id。

```
{ "\u96b4": 7487, "\u772d": 7488, "\u63f7": 7489, "\u596c": 7490, "\u6894": 7491, "\u7e94": 7492, "id2word": [ "<pad>", "<unk>",
< "<sep>", "\u0c22", "\u4f60", "\u6240", "\u505a", "\u7684", "\u4e00", "\u5207", "\u5f00", "\u5fc3", "\u5c31", "\u597d", "\u55ef",
< "\u56e0", "\u4e3a", "\u91cc", "\u53ea", "\u6709", "\u5bb6", "\u4e60", "\u67d0", "\uff0c", "\u8fd8", "\u8fd9", "\u4e2a", "\u7528",
< "\u4eec", "\u5bbf", "\u820d", "\u90fd", "\u662f", "\u4e48", "\u5389", "\u5bb3", "\u4eba", "\u5417", "\u773c", "\u775b", "\u7279",
< "\u522b", "\u641e", "\u7b11", "\u571f", "\u4e5f", "\u4e0d", "\u634f", "\u4f46", "\u89c9", "\u5f97", "\u633a", "\u53ef", "\u7231",
< "\u554a", "\u4eca", "\u5929", "\u70b9", "\u4e86", "\uff1f", "\u6bd4", "\u4e25", "\u91cd", "\u5403", "\u836f", "\u7ba1", "\u53bb",
< "\u6253", "\u9488", "\u3002", "\u62d6", "\u7740", "\u4e0b", "\u8f88", "\u5b50", "\u60f3", "\u8424", "\u706b", "\u866b", "\u592a",
<
```

在id2word列表里，按照id顺序存储了每个字。

清洗情感分类数据

```
# 加载停用词库
def load_stopwords(path):
    with open(path, 'r', encoding='utf-8') as file:
        stopwords = set([line.strip() for line in file.readlines()])
    return stopwords

# 清洗文本，去除停用词和标点符号
def clean_text(text, stopwords):
    # 确保text是字符串类型
    if isinstance(text, str):
        # 去除标点符号
        text = re.sub(pattern=r'[\u3002|\u201c|\u201d|\u201e|\u201f|\u2017|\u2018|\u2019|\u201a|\u201b|\u201c|\u201d|\u201e|\u201f|\u2017|\u2018|\u2019|\u201a|\u201b]', repl='', text)
        # 分词
        words = jieba.cut(text)
        # 去除停用词
        filtered_words = [word for word in words if word not in stopwords and word != ' ']
        return ''.join(filtered_words)
    else:
        return ''

# 加载CSV文件
df = pd.read_csv('emo_less200.csv')

# 加载停用词库
stopwords = load_stopwords('stopwords_full.txt')

# 清洗文本
df['review'] = df['review'].apply(lambda x: clean_text(str(x), stopwords))

# 保存处理后的数据到新的CSV文件
df.to_csv('cleaned_emo.csv', index=False)
```

在这里，对情感分析的数据进行清洗。首先，加载数据和停用词（对情感分析没有用的词），随后去掉文本中的标点和停用词。这样，可以去除无关信息，增加训练效率和模型准确率。

生成训练用数据

生成对话和情感分类任务的训练用数据不同。

这是生成对话任务的部分。

```
class MyDataSet(Data.Dataset):
    # 是类的构造方法，接受一个参数 datas，是数据集的主体，它存储了所有样本的数据
    def __init__(self, datas):
        self.datas = datas

    def __getitem__(self, item):
        data = self.datas[item]
        # 解码器的输入
        decoder_input = data[:-1]
        # 标准输出（去掉第一列是为了和输入对齐）
        decoder_output = data[1:]
        # 计算解码器输入和标准输出的长度
        decoder_input_len = len(decoder_input)
        decoder_output_len = len(decoder_output)
        # 方法的返回值是一个字典，包含了 decoder_input、
        # decoder_input_len、decoder_output 和 decoder_output_len，这些是模型训练或推理时所需要的输入和输出
        return {"decoder_input": decoder_input, "decoder_input_len": decoder_input_len,
                "decoder_output": decoder_output, "decoder_output_len": decoder_output_len}

    def __len__(self):
        return len(self.datas)
```

将训练数据依次转换为ID后，构造数据集类。

通过数据集类的 `__getitem__` 方法，访问每一个数据（即每一组对话）时，都能得到它的解码器输入和应有的解码器输出，以及解码器输入与解码器输出的长度。这在训练中有所应用。

```
def padding_batch(self, batch):
    decoder_input_lens = [d["decoder_input_len"] for d in batch]
    decoder_output_lens = [d["decoder_output_len"] for d in batch]

    decoder_input_maxlen = max(decoder_input_lens)
    decoder_output_maxlen = max(decoder_output_lens)

    for d in batch:
        d["decoder_input"].extend([word2id("<pad>")]*(decoder_input_maxlen-d["decoder_input_len"]))
        d["decoder_output"].extend([word2id("<pad>")]*(decoder_output_maxlen-d["decoder_output_len"]))
    decoder_inputs = torch.tensor([d["decoder_input"] for d in batch], dtype=torch.long)
    decoder_outputs = torch.tensor([d["decoder_output"] for d in batch], dtype=torch.long)

    return decoder_inputs, decoder_outputs
```

数据集类的 `padding_batch` 方法用于对一个批次的数据进行填充。对于每个批次的数据，将解码器输入与输出用 `<pad>` 填充至相同长度（也就是这个批次中的最大长度）。最后，将填充后的结果转换为 tensor，并返回。

这个方法将在使用 `DataLoader` 时作为 `collate_fn` 方法。

```
dataset = MyDataSet(train_num_data)
data_loader = Data.DataLoader(dataset, batch_size=batch_size, collate_fn=dataset.padding_batch)
print(torch.__version__)
print(torch.cuda.is_available())

model = GPT().to(device)
print(model)
```

使用 `DataLoader` 加载数据并填充后，就可以开始搭建GPT模型了。

接下来是情感分类任务的部分。

```

class EmoDataSet(Data.Dataset):
    def __init__(self, datas):
        self.datas = datas

    def __getitem__(self, item):
        sentence = self.datas[item]
        label = sentence[-1]
        decoder_input = sentence[:-2] # 除去最后的label和<sep>
        decoder_output = sentence[1:-1] # 从第二个词开始
        decoder_input_len = len(decoder_input)
        decoder_output_len = len(decoder_output)
        return {
            "decoder_input": decoder_input,
            "decoder_input_len": decoder_input_len,
            "decoder_output": decoder_output,
            "decoder_output_len": decoder_output_len,
            "label": label # 这里返回label
        }

    def __len__(self):

```

在getitem方法中，需要去除最后的label。方法的返回值中，还要返回label。

```

def __len__(self):
    return len(self.datas)

def padding_batch(self, batch):
    decoder_input_lens = [d["decoder_input_len"] for d in batch]
    decoder_output_lens = [d["decoder_output_len"] for d in batch]
    decoder_input_maxlen = max(decoder_input_lens)
    decoder_output_maxlen = max(decoder_output_lens)

    for d in batch:
        d["decoder_input"].extend([word2id["<pad>"]] * (decoder_input_maxlen - d["decoder_input_len"]))
        d["decoder_output"].extend([word2id["<pad>"]] * (decoder_output_maxlen - d["decoder_output_len"]))
    decoder_inputs = torch.tensor([d["decoder_input"] for d in batch], dtype=torch.long)
    decoder_outputs = torch.tensor([d["decoder_output"] for d in batch], dtype=torch.long)
    labels = torch.tensor([d["label"] for d in batch], dtype=torch.long) # 获取标签
    return decoder_inputs, decoder_outputs, labels

```

在padding_batch方法中，还需要获取标签。

搭建模型

搭建点积注意力层

$$\tilde{V} = Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}}) * V$$

计算注意力的公式如图。下面的代码是对这个公式的具体实现，也就是GPT模型中的点积注意力层。

```
class ScaledDotProductAttention(nn.Module):
    def __init__(self, d_k):
        super(ScaledDotProductAttention, self).__init__()
        self.d_k = d_k

    def forward(self, q, k, v, attention_mask):
        # 计算输入的查询 (Q)、键 (K) 和值 (V) 之间的加权关系。
        # 通过计算查询与键的点积来评估它们的相关性，再通过 softmax 计算注意力分布，最后得到基于该分布加权的值向量
        scores = torch.matmul(q, k.transpose(-1, -2)) / np.sqrt(self.d_k)
        scores.masked_fill_(attention_mask, -1e9)
        attn = nn.Softmax(dim=-1)(scores)
        context = torch.matmul(attn, v)
        return context, attn
```

这里定义了类的初始化函数和点积注意力层的前向传播函数。

在前向传播函数中，需要计算Q和K的注意力得分，再对这个分数应用掩码。随后，使用应用掩码后的注意力分数通过 softmax 计算注意力权重，得到最终的上下文向量。

搭建多头注意力层

$$\begin{cases} head_i = Attention(Q_i, K_i, V_i) = softmax(\frac{Q_i K_i^T}{\sqrt{d_k}}) * V, i \in [1, H] \\ MultiHeadAttention(Q, K, V) = Concat(head_0, head_1, \dots, head_H) \end{cases}$$

多头注意力层输出的公式如图所示。

```
2 usages
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads, d_k, d_v):
        super(MultiHeadAttention, self).__init__()
        self.d_model = d_model
        self.n_heads = n_heads
        self.d_k = d_k
        self.d_v = d_v
        self.w_q = nn.Linear(d_model, d_k * n_heads, bias=False)
        self.w_k = nn.Linear(d_model, d_k * n_heads, bias=False)
        self.w_v = nn.Linear(d_model, d_v * n_heads, bias=False)
        self.fc = nn.Linear(n_heads * d_v, d_model, bias=False)
        self.layer_norm = nn.LayerNorm(d_model)

    def forward(self, q, k, v, attention_mask):
        residual, batch_size = q, q.size(0)
        # 将输入的查询、键和值分别映射到多个注意力头
        q = self.w_q(q).view(batch_size, -1, self.n_heads, self.d_k).transpose(1, 2)
        k = self.w_k(k).view(batch_size, -1, self.n_heads, self.d_k).transpose(1, 2)
        v = self.w_v(v).view(batch_size, -1, self.n_heads, self.d_v).transpose(1, 2)
        attention_mask = attention_mask.unsqueeze(1).repeat(1, self.n_heads, 1, 1)
        # 并行计算每个头的注意力，最终通过全连接层将多头的结果合并起来。
        context, attn = ScaledDotProductAttention(self.d_k)(q, k, v, attention_mask)
        context = context.transpose(1, 2).reshape(batch_size, -1, self.n_heads * self.d_v)
        output = self.fc(context)
        return self.layer_norm(output + residual), attn
```

在前向传播方法中，首先将q,k,v映射到各个注意力头上，计算每个注意力头上的自注意力，再将它们拼接起来，通过全连接层得到最后的输出。

搭建前馈神经网络层

```
2 usages
class PoswiseFeedForwardNet(nn.Module):
    def __init__(self, d_model, d_ff):
        super(PoswiseFeedForwardNet, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(d_model, d_ff, bias=False),
            nn.ReLU(),
            nn.Linear(d_ff, d_model, bias=False)
        )
        self.layernorm = nn.LayerNorm(d_model)

    def forward(self, inputs):
        # 首先将输入经过fc网络，随后应用残差连接和层归一化，得到输出。
        residual = inputs
        output = self.fc(inputs)
        return self.layernorm(output + residual)
```

前馈神经网络层由两个线性全连接层和中间的ReLU激活函数组成。在它的前向传播方法中，首先将输入经过fc网络，随后应用残差连接和层归一化，得到输出。

生成注意力掩码

```
# 生成一个注意力掩码（Attention Mask），用来处理填充（padding）部分的注意力。
# 在序列数据处理中，填充部分（通常是0）不应该对模型的计算产生影响，因此需要通过掩码将其屏蔽掉
1 usage
def get_attn_pad_mask(seq_q, seq_k):
    """
    seq_q: [batch_size, seq_len]
    seq_k: [batch_size, seq_len]
    seq_len could be src_len or it could be tgt_len
    seq_len in seq_q and seq_len in seq_k maybe not equal
    """
    batch_size, len_q = seq_q.size()
    batch_size, len_k = seq_k.size()
    # eq(zero) is PAD token
    pad_attn_mask = seq_k.data.eq(0).unsqueeze(1)  # [batch_size, 1, len_k], True is masked
    return pad_attn_mask.expand(batch_size, len_q, len_k)  # [batch_size, len_q, len_k]
```

对输入序列 `seq_q` 和 `seq_k` 中的填充位置生成布尔掩码，并对它进行扩展，生成一个形状为 `[batch_size, len_q, len_k]` 的掩码张量。

生成子序列注意力掩码

```
def get_attn_subsequence_mask(seq):
    # seq 是一个张量，形状为 [batch_size, tgt_len]，即一个批次中每个序列的目标长度。
    # batch_size 是批次大小，tgt_len 是目标序列的长度（可能是解码过程中的时间步长）。
    """
    seq: [batch_size, tgt_len]
    """
    # attn_shape 是生成注意力掩码时需要的张量形状，形状为 [batch_size, tgt_len, tgt_len]。
    # seq.size(0) 是批次大小 (batch_size)。
    # seq.size(1) 是目标序列的长度 (tgt_len)。
    attn_shape = [seq.size(0), seq.size(1), seq.size(1)]
    # 成一个上三角矩阵，并将对角线及以下部分的值设为 0。参数 k=1 表示将对角线以上（包括对角线本身）的部分设为 1，其余部分设为 0。
    subsequence_mask = np.triu(np.ones(attn_shape), k=1) # Upper triangular matrix
    # numpy 数组转换为 PyTorch 张量，并使用 .byte() 方法将其转换为 uint8 类型（即 torch.uint8），这是因为掩码通常以 0 和 1 表示。
    subsequence_mask = torch.from_numpy(subsequence_mask).byte()
    # 移动到指定的设备上
    subsequence_mask = subsequence_mask.to(device)
    # 返回形状为 [batch_size, tgt_len, tgt_len] 的掩码张量，它表示每个时间步对其他时间步的可见性
    return subsequence_mask # [batch_size, tgt_len, tgt_len]
```

生成一个上三角掩码，防止解码时看到未来的时间步。

搭建解码层

```
class DecoderLayer(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, d_k, d_v):
        super(DecoderLayer, self).__init__()
        self.attention = MultiHeadAttention(d_model, n_heads, d_k, d_v)
        self.pos_ffn = PoswiseFeedForwardNet(d_model, d_ff)

    def forward(self, inputs, attention_mask):
        # 将输入通过多头自注意力层和前馈神经网络层，得到输出。
        outputs, self_attn = self.attention(inputs, inputs, inputs, attention_mask)
        outputs = self.pos_ffn(outputs)
        return outputs, self_attn
```

解码层由一个多头注意力层和一个前馈神经网络层组成。

它的forward方法是：将输入通过多头自注意力层和前馈神经网络层，得到输出。

搭建解码器

对话任务和情感分析任务的解码器部分有所不同。

对话任务的解码器部分：

```

class Decoder(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, d_k, d_v, vocab_size, max_pos, n_layers):
        super(Decoder, self).__init__()
        self.tgt_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Embedding(max_pos, d_model)
        self.layers = nn.ModuleList([DecoderLayer(d_model, n_heads, d_ff, d_k, d_v) for _ in range(n_layers)])

    def forward(self, dec_inputs):
        # 首先生成位置编码，得到加了位置信息的嵌入结果 dec_outputs。
        # 随后，生成填充掩码和注意力掩码，并将它们组合起来。
        seq_len = dec_inputs.size(1)
        pos = torch.arange(seq_len, dtype=torch.long, device=device)
        pos = pos.unsqueeze(0).expand_as(dec_inputs)

        dec_outputs = self.tgt_emb(dec_inputs) + self.pos_emb(pos)

        dec_self_attn_pad_mask = get_attn_pad_mask(dec_inputs, dec_inputs)
        dec_self_attn_subsequence_mask = get_attn_subsequence_mask(dec_inputs)
        dec_self_attn_mask = torch.gt((dec_self_attn_pad_mask + dec_self_attn_subsequence_mask), other: 0)
        dec_self_attns = []
        # 解码层共有n_layers个。
        # 对于每个解码层，将输入放入解码层进行处理。
        # 解码层中的输入是加了位置编码的输入，
        # 解码层中的掩码是在之前组合起来的填充掩码和注意力掩码。
        # 每一层的输出会变成下一层的输入。
        for layer in self.layers:
            dec_outputs, dec_self_attn = layer(dec_outputs, dec_self_attn_mask)
            dec_self_attns.append(dec_self_attn)
        return dec_outputs, dec_self_attns

```

在前向传播方法中，首先生成位置编码，得到加了位置信息的嵌入结果 `dec_outputs`。随后，生成填充掩码和注意力掩码，并将它们组合起来。

解码层共有 `n_layers` 个。对于每个解码层，将输入放入解码层进行处理。解码层中的输入是加了位置编码的输入，解码层中的掩码是在之前组合起来的填充掩码和注意力掩码。每一层的输出会变成下一层的输入。

情感分析任务的解码器部分：

```

class Decoder(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, d_k, d_v, vocab_size, max_pos, n_layers, use_mask=True):
        super(Decoder, self).__init__()
        self.tgt_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Embedding(max_pos, d_model)
        self.layers = nn.ModuleList([DecoderLayer(d_model, n_heads, d_ff, d_k, d_v) for _ in range(n_layers)])
        self.use_mask = use_mask # 控制是否使用mask

    def forward(self, dec_inputs):
        """
        dec_inputs: [batch_size, tgt_len]
        """
        seq_len = dec_inputs.size(1)
        pos = torch.arange(seq_len, dtype=torch.long, device=device)
        pos = pos.unsqueeze(0).expand_as(dec_inputs) # [seq_len] -> [batch_size, seq_len]

        dec_outputs = self.tgt_emb(dec_inputs) + self.pos_emb(pos) # [batch_size, tgt_len, d_model]

        if self.use_mask:
            dec_self_attn_pad_mask = get_attn_pad_mask(dec_inputs, dec_inputs) # [batch_size, tgt_len, tgt_len]
            dec_self_attn_subsequence_mask = get_attn_subsequence_mask(dec_inputs) # [batch_size, tgt_len, tgt_len]
            dec_self_attn_mask = torch.gt((dec_self_attn_pad_mask + dec_self_attn_subsequence_mask),
                                           other: 0) # [batch_size, tgt_len, tgt_len]
        else:
            # 如果禁用mask，创建一个全为False的mask
            dec_self_attn_mask = torch.zeros((dec_inputs.size(0), seq_len, seq_len), dtype=torch.bool, device=device)

        dec_self_attns = []
        for layer in self.layers:
            dec_outputs, dec_self_attn = layer(dec_outputs, dec_self_attn_mask)
            dec_self_attns.append(dec_self_attn)
        return dec_outputs, dec_self_attns

```

在情感分类任务中，有使用mask和禁用mask的选项。如果使用mask，就按照和对话任务相同的方式生成mask；否则，生成一个全是 false 的mask。这样，mask就被禁用，不会有位置被屏蔽。

搭建GPT模型

生成对话任务和情感分类任务的GPT模型有差别。

这是生成对话任务的GPT模型。

```
class GPT(nn.Module):
    def __init__(self):
        super(GPT, self).__init__()
        # 一个decoder模块和一个将解码器输出映射到词表的线性层
        self.decoder = Decoder(d_model, n_heads, d_ff, d_k, d_v, vocab_size, max_pos, n_layers)
        self.projection = nn.Linear(d_model, vocab_size)

    def forward(self, dec_inputs):
        # 输入通过Decoder模块得到解码器的输出和自注意力权重，
        # 再通过线性层得到每个位置上词汇表的概率分布。
        dec_outputs, dec_self_attns = self.decoder(dec_inputs)
        dec_logits = self.projection(dec_outputs)
        return dec_logits.view(-1, dec_logits.size(-1)), dec_self_attns
```

GPT模型的 `__init__` 方法定义了一个decoder模块和一个将解码器输出映射到词表的线性层。

它的前向传播方法要求输入通过 `Decoder` 模块得到解码器的输出和自注意力权重，再通过线性层得到每个位置上词汇表的概率分布。

```
usage
def greedy_decoder(self, dec_input):
    # 不断循环直到terminal变为true。
    # 将输入放入解码器获得输出，
    # 再通过线性投影层获得词表的概率分布，
    # 随后获取概率最大的词，
    # 并将这个词作为输入拼接到输入序列中。
    # 最终返回生成的序列。
    terminal = False
    start_dec_len = len(dec_input[0])
    while not terminal:
        # 如果遇到 '<sep>' 或生成的长度超过100，就终止。
        if len(dec_input[0]) - start_dec_len > 100:
            next_symbol = word2id['<sep>']
            dec_input = torch.cat([dec_input.detach(), torch.tensor([next_symbol], dtype=dec_input.dtype, device=device)], dim=0)
            break
        dec_outputs, _ = self.decoder(dec_input)
        projected = self.projection(dec_outputs)
        prob = projected.squeeze(0).max(dim=-1, keepdim=False)[1]
        next_word = prob.data[-1]
        next_symbol = next_word
        if next_symbol == word2id["<sep>"]:
            terminal = True
        dec_input = torch.cat([dec_input.detach(), torch.tensor([next_symbol], dtype=dec_input.dtype, device=device)], dim=0)
    return dec_input
```

贪心解码方法是基于贪心策略的。在方法中，不断循环直到terminal变为true。每次，将输入放入解码器获得输出，再通过线性投影层获得词表的概率分布，随后获取概率最大的词，并将这个词作为输入拼接到输入序列中，最终返回生成的序列。如果遇到 `<sep>` 或生成的长度超过100，就将terminal置为true。

```

# usage
def answer(self, sentence):
    # 首先将输入的句子转换成词表的形式 (word2id)，并转换为张量。
    # 调用greedy_decoder生成输出序列，再通过id2word转换成单词。
    # 提取`<sep>`符之间的部分作为回答，将回答拼接成字符串输出。
    dec_input = [word2id.get(word, 1) if word != '\t' else word2id['<sep>'] for word in sentence]
    dec_input = torch.tensor(dec_input, dtype=torch.long, device=device).unsqueeze(0)

    output = self.greedy_decoder(dec_input).squeeze(0)
    out = [id2word[int(id)] for id in output]
    sep_indexes = []
    for i in range(len(out)):
        if out[i] == "<sep>":
            sep_indexes.append(i)
    answer = out[sep_indexes[-2] + 1:-1]
    answer = "".join(answer)
    return answer

```

answer方法首先将输入的句子转换成词表的形式 (word2id)，并转换为张量。随后，调用greedy_decoder生成输出序列，再通过id2word转换成单词。提取 <sep> 符之间的部分作为回答，将回答拼接成字符串输出。

```

model.add_state_dict(torch.load('GPT.pt')['model_state_dict'], strict=False)
GPT(
  (decoder): Decoder(
    (tgt_emb): Embedding(7493, 768)
    (pos_emb): Embedding(1800, 768)
    (layers): ModuleList(
      (0-5): 6 x DecoderLayer(
        (attention): MultiHeadAttention(
          (w_q): Linear(in_features=768, out_features=512, bias=False)
          (w_k): Linear(in_features=768, out_features=512, bias=False)
          (w_v): Linear(in_features=768, out_features=512, bias=False)
          (fc): Linear(in_features=512, out_features=768, bias=False)
          (layernorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        )
        (pos_ffn): PoswiseFeedForwardNet(
          (fc): Sequential(
            (0): Linear(in_features=768, out_features=2048, bias=False)
            (1): ReLU()
            (2): Linear(in_features=2048, out_features=768, bias=False)
          )
          (layernorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        )
      )
    )
  )
  (projection): Linear(in_features=768, out_features=7493, bias=True)
)

```

```

v # 参数
# 输入序列的最大长度
max_pos = 1800
# 模型的嵌入维度
d_model = 768
# 前馈神经网络层的维度
d_ff = 2048
# 注意力机制中键（Key）、查询（Query）和值（Value）的维度
d_k = d_v = 64
# 编码器（Encoder）或解码器（Decoder）的层数
n_layers = 6
# 多头注意力机制中的头数
n_heads = 8 |
CLIP = 1

```

对话生成任务的GPT模型结构和参数如图所示。

下面是情感分类任务的GPT模型。

```

v class GPT(nn.Module):
v     def __init__(self, use_mask=True):
v         super(GPT, self).__init__()
v         self.decoder = Decoder(d_model, n_heads, d_ff, d_k, d_v, vocab_size, max_pos, n_layers, use_mask)
v         self.projection = nn.Linear(d_model, vocab_size)
v         self.classifier = nn.Linear(d_model, out_features=2) # classify task

v     def forward(self, dec_inputs, classify=False):
v         # dec_inputs: [batch_size, tgt_len]
v         # classify: 是否进行分类任务（False 表示生成任务）
v         dec_outputs, dec_self_attns = self.decoder(dec_inputs) # [batch_size, tgt_len, d_model]

v         if classify:
v             # 分类任务：取最后一个时间步的输出进行分类
v             last_token_output = dec_outputs[:, -1, :] # [batch_size, d_model]
v             class_logits = self.classifier(last_token_output) # [batch_size, 2]
v             return class_logits, dec_self_attns
v         else:
v             # 生成任务：使用投影层生成词汇分布
v             dec_logits = self.projection(dec_outputs) # [batch_size, tgt_len, vocab_size]
v             return dec_logits.view(-1, dec_logits.size(-1)), dec_self_attns

```

相比对话生成任务，这里的GPT模型多了一层线性分类层。

在前向传播方法中，如果在执行分类任务，就要取出解码器输出序列中的最后一个时间步 `last_token_output`，将其通过 `self.classifier` 进行分类，得到一个二分类的结果。

```

def greedy_decoder(self, dec_input):

    terminal = False
    start_dec_len = len(dec_input[0])
    # 一直预测下一个单词，直到预测到"<sep>"结束，如果一直不到"<sep>"，则根据长度退出循环，并在最后加上"<sep>"字符
    while not terminal:
        if len(dec_input[0]) - start_dec_len > 100:
            next_symbol = word2id['<sep>']
            dec_input = torch.cat(
                tensors=[dec_input.detach(), torch.tensor( data: [[next_symbol]], dtype=dec_input.dtype, device=device)], -1)
            break
        dec_outputs, _ = self.decoder(dec_input)
        projected = self.projection(dec_outputs)
        prob = projected.squeeze(0).max(dim=-1, keepdim=False)[1]
        next_word = prob.data[-1]
        next_symbol = next_word
        if next_symbol == word2id["<sep>"]:
            terminal = True

        dec_input = torch.cat(
            tensors=[dec_input.detach(), torch.tensor( data: [[next_symbol]], dtype=dec_input.dtype, device=device)], -1)

    return dec_input

```

```

def answer(self, sentence):
    # 把原始句子的\t替换成"<sep>"
    dec_input = [word2id.get(word, 1) if word != '\t' else word2id['<sep>'] for word in sentence]
    dec_input = torch.tensor(dec_input, dtype=torch.long, device=device).unsqueeze(0)

    output = self.greedy_decoder(dec_input).squeeze(0)
    out = [id2word[int(id)] for id in output]
    # 统计"<sep>"字符在结果中的索引
    sep_indexes = []
    for i in range(len(out)):
        if out[i] == "<sep>":
            sep_indexes.append(i)

    # 取最后两个sep中间的内容作为回答

    answer = out[sep_indexes[-2] + 1:-1]

    answer = "".join(answer)
    return answer

```

二者的 greedy_decoder 和 answer 方法相同。

对话生成任务

模型训练

```

def train_step(model, data_loader, optimizer, criterion, clip=1, print_every=None):
    # 设置模型为训练模式
    model.train()

    if print_every == 0:
        print_every = 1

    print_loss_total = 0 # 每次打印都重置

    epoch_loss = 0

# dec_inputs 和 dec_outputs, 分别是解码器输入和输出
# 按批次 (batch) 加载数据
# tqdm 是用于显示进度条
for i, (dec_inputs, dec_outputs) in enumerate(tqdm(data_loader)):
    ...

    dec_inputs: [batch_size, tgt_len]
    dec_outputs: [batch_size, tgt_len]
    ...

    # 将优化器的梯度清零
    optimizer.zero_grad()
    # 解码器输入和输出转移到合适的设备
    dec_inputs, dec_outputs = dec_inputs.to(device), dec_outputs.to(device)
    # outputs: [batch_size * tgt_len, tgt_vocab_size]
    # 前向传播
    # outputs 是模型的输出, 形状通常是 [batch_size, tgt_len, vocab_size], 即预测的每个位置的词汇概率分布
    outputs, dec_self_attns = model(dec_inputs)
    # 计算损失
    loss = criterion(outputs, dec_outputs.view(-1))
    # 累加损失

```

```

    for i, (dec_inputs, dec_outputs) in enumerate(tqdm(data_loader)):
        ...

        dec_inputs: [batch_size, tgt_len]
        dec_outputs: [batch_size, tgt_len]
        ...

        # 将优化器的梯度清零
        optimizer.zero_grad()
        # 解码器输入和输出转移到合适的设备
        dec_inputs, dec_outputs = dec_inputs.to(device), dec_outputs.to(device)
        # outputs: [batch_size * tgt_len, tgt_vocab_size]
        # 前向传播
        # outputs 是模型的输出, 形状通常是 [batch_size, tgt_len, vocab_size], 即预测的每个位置的词汇概率分布
        outputs, dec_self_attns = model(dec_inputs)
        # 计算损失
        loss = criterion(outputs, dec_outputs.view(-1))
        # 累积损失
        print_loss_total += loss.item()
        epoch_loss += loss.item()
        # 反向传播
        loss.backward()
        # 梯度裁剪
        torch.nn.utils.clip_grad_norm_(model.parameters(), clip)
        # 使用反向传播计算的梯度来更新模型的权重
        optimizer.step()
        # 打印损失信息
        if print_every and (i + 1) % print_every == 0:
            print_loss_avg = print_loss_total / print_every
            print_loss_total = 0
            print('\tCurrent Loss: %.4f' % print_loss_avg)

    # 返回平均损失
    return epoch_loss / len(data_loader)

```



```

def train(model, data_loader):
    # 定义损失函数
    criterion = nn.CrossEntropyLoss(ignore_index=0).to(device)
    # 设置学习率
    optimizer = optim.Adam(model.parameters(), lr=1e-4)
    start_epoch = 0

    if os.path.exists('GPT2.pt'):
        checkpoint = torch.load('GPT2.pt')
        model.load_state_dict(checkpoint['model_state_dict'])
        optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
        start_epoch = checkpoint['epoch'] # 恢复epoch

    for epoch in range(start_epoch, epochs):
        start_time = time.time()
        # 训练
        train_loss = train_step(model, data_loader, optimizer, criterion, CLIP, print_every=10)
        end_time = time.time()
        # 保存训练参数
        if epoch % 3 == 0:
            torch.save({
                'epoch': epoch + 1, # 保存当前epoch
                'model_state_dict': model.state_dict(),
                'optimizer_state_dict': optimizer.state_dict(),
            }, f'GPT2.pt')

        epoch_mins, epoch_secs = epoch_time(start_time, end_time)
        # 打印轮数和这一轮训练的用时
        print(f'Epoch: {epoch + 1:02} | Time: {epoch_mins}m {epoch_secs}s')
        print(f'\tTrain Loss: {train_loss:.3f}')

```

在对话任务的模型分类中，使用adam优化器。训练开始前先检查是否有训练过的模型和优化器参数，如果有，先加载之前的数据，再进行训练。

训练时，每一步使用交叉熵损失函数，反相传播计算梯度，并根据计算的梯度更新模型的权重。每训练三个epoch，保存一次模型和优化器参数。

总共训练了30个epoch，大约用时25小时。

模型测试

在测试时，先加载模型，然后就可以和模型进行对话了。

我：你好

GPT：你好

我：你是谁？

GPT：我是你爸爸

我：好久不见

GPT：好久不见

我：你想和我一起去散步吗

GPT：我想和你一起散散心

我：你最近怎么样？

GPT：我还好

我：我在准备期末考试，很忙

GPT：我也很忙，你呢？

我：我也很忙

GPT：你考的哪个学校？

我：中山大学

GPT：我也是

我：你喜欢中山大学吗

GPT：我喜欢中山大学

我：我们可以一起去中山大学的校园里散步吗

GPT：我们是中山大学的

我：是的

GPT：好啊，我也是

我：你好

GPT：你好

我：你是高中生吗

GPT：不是

我：我们可以一起组乐队吗

GPT：可以啊

我：我们可以一起组一辈子乐队吗

GPT：可以啊

我：你听过《春日影》这首歌吗

GPT：我喜欢的歌词

我：你会唱吗

GPT：我会

我：你可以唱给我听吗

GPT：我会唱

我：你喜欢唱歌吗

GPT：喜欢

我：那太好了

GPT：我喜欢唱歌

我：再见

GPT：我不喜欢你了

我：你觉得玩乐队不开心吗

GPT：我不会

我：那你为什么不喜欢我

GPT：我喜欢你

我：好的

GPT：我喜欢你

我: 你好
GPT: 你好
我: 今天天气真好
GPT: 我们这里天气好
我: 我这里天气也好
GPT: 我这里天气也好
我: 这种天气适合出去玩
GPT: 是的, 天气也好
我: 我想出去吃火锅了
GPT: 我也是
我: 我们可以一起去吃火锅吗
GPT: 好呀好呀
我: 我们可以在地铁站见面吗
GPT: 好呀好呀
我: 我很开心能和你一起出去
GPT: 我也很开心
我: 我很喜欢和你在一起
GPT: 我也很喜欢
我: 我先走了
GPT: 走吧走吧走吧走吧走吧走吧走吧走吧走吧

效果分析

从测试中可以看出, 模型可以理解简单的句子, 比如“你好”, “你是谁”, 以及简单的疑问句。但是, 句子的语法一旦变得复杂 (比如“你听过《春日影》这首歌吗”), 模型就不能给出合理的回答。

情感分析任务

模型训练

```
for i, (dec_inputs, dec_outputs, labels) in enumerate(tqdm(data_loader)):  
    optimizer.zero_grad()  
    dec_inputs, dec_outputs, labels = dec_inputs.to(device), dec_outputs.to(device), labels.to(device)  
    outputs, dec_self_attns = model(dec_inputs, classify=True) # 获取模型输出 logits  
    # 计算交叉熵损失, 输出 logits 和标签  
    loss = criterion(outputs, labels)  
    print_loss_total += loss.item()  
    epoch_loss += loss.item()  
    loss.backward()  
    torch.nn.utils.clip_grad_norm_(model.parameters(), clip)  
    optimizer.step()  
  
    if print_every and (i + 1) % print_every == 0:  
        print_loss_avg = print_loss_total / print_every  
        print_loss_total = 0  
        print(f'\tCurrent Loss: {print_loss_avg:.4f}')
```

return epoch_loss / len(data_loader)

在每一步训练中，使用交叉熵损失函数计算输出标签和实际标签的loss。其余部分与生成对话任务相同。

```
def train(model, data_loader):
    # 处理数据不平衡
    class_counts = df['label'].value_counts().to_dict()
    total_samples = sum(class_counts.values())
    class_weights = {label: total_samples / count for label, count in class_counts.items()}
    weights = torch.tensor([class_weights[0], class_weights[1]], dtype=torch.float).to(device)
    criterion = nn.CrossEntropyLoss(weight=weights).to(device)

    # 设置学习率
    optimizer = torch.optim.Adam([
        {'params': model.classifier.parameters(), 'lr': 1e-3}, # 仅训练分类头
        {'params': model.decoder.layers.parameters(), 'lr': 1e-5} # 微调解码器所有层
        {'params': model.decoder.layers[-1:].parameters(), 'lr': 1e-5} # 微调解码器最后一层
    ])

    for epoch in range(epochs):
        start_time = time.time()
        # 训练
        train_loss = train_step(model, data_loader, optimizer, criterion, CLIP, print_every=10)
        end_time = time.time()
        # 保存训练参数
        torch.save(model.state_dict(), f'GPT2_emo_last_banmask.pt')

        epoch_mins, epoch_secs = epoch_time(start_time, end_time)
        # 打印轮数和这一轮训练的用时
        print(f'Epoch: {epoch + 1:02} | Time: {epoch_mins}m {epoch_secs}s')
        print(f'\tTrain Loss: {train_loss:.3f}')
```

在训练函数中，首先需要处理数据不平衡的情况。

在数据集中，不同类的数据数量相差可能较大。这时，模型可能无法学习到少数类的有效特征，导致模型在训练时无法有效地做出区分。同时，由于一类的数据数量远大于另一类，模型在预测时可能会倾向于把更多测试数据分到多数类中。这样，虽然模型的整体准确率看起来很高，但实际效果可能很差，尤其是在判断少数类的时候。

随后，设置优化器。在这里，我们对分类器头部和解码器使用了不同的学习率进行训练。分类器头部使用的学习率较高，这样方便模型尽快调整到和任务适应的状态。解码器部分的学习率较低，以保持模型输出的稳定性。

在训练时，分别设置仅训练解码器最后一层和训练整个解码器。

模型测试

```
class TestDataset(Dataset):
    """
    测试数据集类。
    """

    def __init__(self, reviews, labels, word2id):
        self.reviews = reviews
        self.labels = labels
        self.word2id = word2id

    def __len__(self):
        return len(self.reviews)

    def __getitem__(self, index):
        review = self.reviews[index]
        label = self.labels[index]

        # 转换文本为 ID
        input_ids = [self.word2id.get(word, self.word2id['<unk>']) if word != ' ' else self.word2id['<sep>']
                     for word in review]
        input_ids += [self.word2id['<sep>']]

        return input_ids, label
```

```
@staticmethod
def collate_fn(batch):
    inputs, labels = zip(*batch)
    max_len = max(len(seq) for seq in inputs)

    # padding
    padded_inputs = [seq + [word2id['<pad>']] * (max_len - len(seq)) for seq in inputs]
    inputs_tensor = torch.tensor(padded_inputs, dtype=torch.long)
    labels_tensor = torch.tensor(labels, dtype=torch.long)
    return inputs_tensor, labels_tensor
```

测试数据集类和训练数据集类有所不同。

这里的 `getitem` 方法会返回文本的id序列和标签。

`collate_fn` 方法会将id序列和labels分开，分别放进不同的列表中。然后，对每个id序列进行填充，转换为pytorch张量。

```
def predict_with_batches(model, data_loader, device):
    model.eval()
    all_predictions = []

    with torch.no_grad():
        for inputs, _ in data_loader:
            inputs = inputs.to(device)
            outputs, _ = model(inputs, classify=True)
            predictions = torch.argmax(outputs, dim=1).tolist()
            all_predictions.extend(predictions)

    return all_predictions

def calculate_accuracy(predictions, labels):
    # 计算预测结果的准确率
    correct = sum(p == l for p, l in zip(predictions, labels))
    return correct / len(labels)
```

这两个方法会进行预测并计算准确率。

`predict_with_batches` 对dataloader中的每一个batch，通过模型进行前向传播得到输出，使用最大值索引得到预测类别，将预测结果添加到预测列表。

`calculate_accuracy` 计算预测类别和实际类别相同的数据个数，除以数据总数，得到准确率。

```

if __name__ == "__main__":
    # 加载微调后的模型
    device = torch.device('cuda')
    model = GPT().to(device)
    model.load_state_dict(torch.load(f'GPT2_emo_last_banmask.pt', weights_only=True))

    # 加载测试数据
    test_file = "emo_less300.csv"
    dataframe = pd.read_csv(test_file)
    df = dataframe.sample(frac=1, random_state=18).reset_index(drop=True)
    reviews = df['review'].astype(str).tolist()
    labels = df['label'].astype(int).tolist()

    # 创建数据集和 DataLoader
    batch_size = 8
    dataset = TestDataset(reviews, labels, word2id)
    data_loader = DataLoader(dataset, batch_size=batch_size, collate_fn=TestDataset.collate)

    # 分批预测
    predictions = predict_with_batches(model, data_loader, device)

    # 计算准确率
    accuracy = calculate_accuracy(predictions, labels)
    print(f"测试集准确率: {accuracy * 100:.2f}%")

```

这是测试模型部分的main函数。

整体的测试步骤是：

1. 创建模型实例
2. 加载模型
3. 加载测试数据
4. 生成文本和标签列表
5. 创建dataloader实例。
6. 分批预测。
7. 计算预测准确率。

效果分析

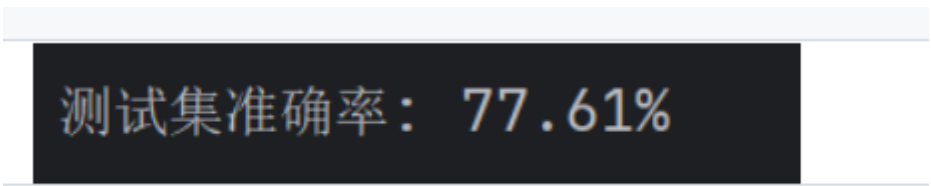
训练全部层，不禁用mask的效果：

测试集准确率： 79.87%

训练全部层，训练15个epoch，禁用mask的效果：

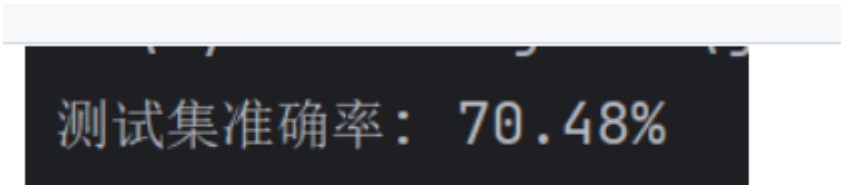
测试集准确率： 75.67%

训练全部层，训练30个epoch，禁用mask的效果：



测试集准确率： 77.61%

只训练最后一层，不禁用mask的效果：



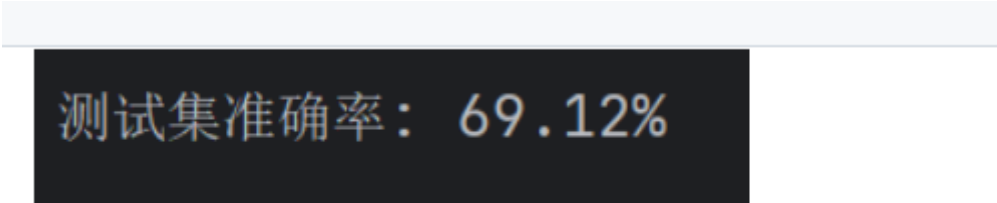
测试集准确率： 70.48%

只训练最后一层，训练15个epoch，禁用mask 的效果：



测试集准确率： 70.48%

只训练最后一层，训练30个epoch，禁用mask 的效果：



测试集准确率： 69.12%

可以看出，在训练15个epoch时，训练所有层且使用mask得到的准确率，大于训练所有层且禁用mask得到的准确率，大于仅训练最后一层且使用mask得到的准确率，等于仅训练最后一层且禁用mask得到的准确率。将epoch增加到30，训练所有层的准确率上升了，而只训练最后一层和输出层的反而降低了。

文本语义相似度判断

要实现文本语义相似度判断，需要从数据预处理、模型结构、训练函数三个方面进行微调。

首先是数据预处理部分。如果希望得到二分类结果，需要对每一组文本打上标签，这个标签说明这组文本是否相似。如果希望得到相似性评分，那么就要在标签中对一组文本的相似性进行评分。

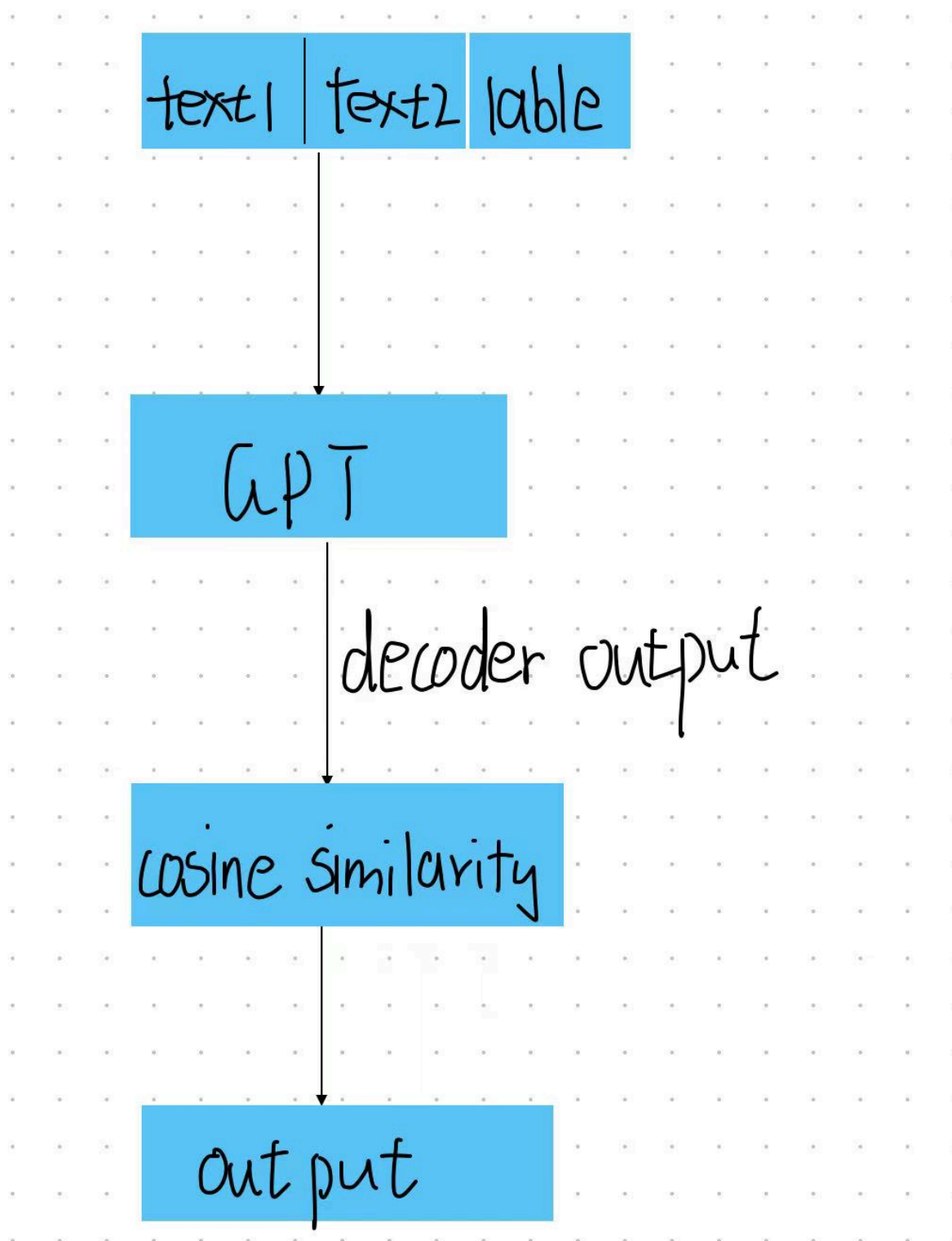
然后是模型结构部分。在transformer中，每个token都是一个向量，token的不同维度表示不同的信息，含义相似的token的方向也相近。所以，我们需要计算两个文本之间的向量表示，再计算相似性分数，从而判断两个文本是否相似。

在已有模型的基础上，可以使用解码器的输出作为句子的向量表示。获得一组句子的向量表示后，可以使用余弦相似度、欧式距离等方法计算相似性分数。

余弦相似度的计算公式如图。

$$\text{Cosine Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{|\mathbf{A}| |\mathbf{B}|}$$

模型结构如图：



在训练时，如果希望得到二分类结果，就使用二元交叉熵损失。如果想得到相似性评分，就使用均方误差作为损失函数。

实验心得

本次大作业是一个难度很大的任务。刚拿到作业题目时，我和队友完全是茫然的，不知道怎么入手。我们花了一周多的时间研究transformer架构，弄清楚它的原理、组成和作用。

根据作业题目中的提示，我们完成了对话生成任务的数据处理部分。首先，考虑到电脑性能，我们过滤掉了长度超过200的数据。随后，我们用剩下的数据生成了word2id字典和id2word列表。

在搭建模型时，我们使用了6个由多头注意力层和前馈神经网络层组成的decoder层。在生成输出时，我们使用了一个线性层映射词表，使用了贪心解码方法获得输出的单词。

在模型训练和测试中，我们使用了Adam优化器，结合交叉熵损失函数，优化参数、计算损失。对话任务的模型总共训练了30个epoch，大约用时25小时。

在情感分类的部分，首先在数据集处理，考虑电脑性能，我们过滤掉了超过200的数据。然后，为了让分类任务的准确率更高，我们简单删除了停止词，同时还删除了标点符号。

接着我们在对话生成的模型基础上，加入了分类头来返回分类任务的结果。然后还设置了是否禁用mask的选项。

然后在训练的部分，首先在句子与句子之间加上 `<sep>`。然后处理数据与word2id的映射，如果出现的字在word2id有映射就使用对应的值，反之就划为未知符号。然后为了防止数据集中正面评价和负面评价数量差距大导致模型不准确，将两个标签的权重加入损失的计算。然后通过设置参数来控制训练所有层还是最后一层加输出层。因为情感分类的数据集大小比对话生成的小很多，所以这里只训练15个epoch。

最后是比较结果，我们这里的测试集是用过滤了超过300的数据集，并使用与训练时不同的随机种子打乱数据集。发现训练所有层的准确率会比只训练最后一层加输出层更高。而禁用mask的会比各自的没禁用时的会低一些。然后我们加大了epoch，发现在epoch为30的时候，训练所有层的准确率上升了，而只训练最后一层和输出层的反而降低了。